Reduced-ML-KEM Cryptanalysis: Parameter Scaling Cryptanalysis of Module-LWD

1. MLWE Problem Formulation & Instance Generation

Description & Details: This snippet defines the target cryptosystem. The ToyMLWE class generates the public matrix A (constructed as a negacyclic Toeplitz block matrix to respect polynomial multiplication in R_q) and samples the secret s and error e from the Centered Binomial Distribution (CBD). The public key t is computed as t=As+e (mod q). This establishes the algebraic Module-LWE instance that the subsequent lattice attacks will target.

Python

```python
def sample_cbd(eta, size, rng):
    """Centered binomial distribution CBD_eta."""
    if eta == 0:
        return np.zeros(size, dtype=np.int64)
    a = rng.integers(0, 2, size=(size, eta))
    b = rng.integers(0, 2, size=(size, eta))
    return (a.sum(axis=1) - b.sum(axis=1)).astype(np.int64)

class ToyMLWE:
    # ... (initialization omitted for brevity)
    def keygen(self):
        n, k, q = self.n, self.k, self.q
        A_polys = [[self._random_poly() for _ in range(k)] for
_ in range(k)]
        A_block = self._build_A_block(A_polys,
transpose=False)
        s_polys = [self._small_poly() for _ in range(k)]
        e_polys = [self._small_poly() for _ in range(k)]
```

```
        s_flat = np.concatenate(s_polys)
        e_flat = np.concatenate(e_polys)
        t_flat = (A_block @ s_flat + e_flat) % q
        pk = {"A_polys": A_polys, "A_block": A_block, "t":
t_flat}
        sk = {"s": s_flat, "e": e_flat, "s_polys": s_polys}
        return pk, sk
```

2. Lattice Embedding (Kannan's Construction)

Description & Details: To apply geometric lattice reduction to MLWE, the algebraic equation b=As+e (mod q) is mapped into a lattice using Kannan's embedding. The matrix qI_m accounts for the modular arithmetic, while the appended 1 in the bottom-right corner scales the target vector. A short vector in this lattice takes the form (e,-s,1), directly yielding the secret key if found.

Python

```python
def build_embedding_lattice(A, b, q):
    """ Constructs the Kannan embedding basis B for LWE.
    B = [ A^T | I_n | 0 ]
        [ q*I_m | 0 | 0 ]
        [ b | 0 | 1 ]
    Returns an fpylll IntegerMatrix. """
    m, n = A.shape
    d = m + n + 1
    B = IntegerMatrix(d, d)
    for i in range(n):
        for j in range(m):
            B[i, j] = int(A[j, i])
```

```
            B[i, m + i] = 1
    for i in range(m):
        B[n + i, i] = int(q)
    for j in range(m):
        B[d - 1, j] = int(b[j])
    B[d - 1, d - 1] = 1
    return B
```

3. Hybrid Attack: Guess-and-Reduce

Description & Details: This implements the hybrid attack, which trades dimension for combinatorial complexity. It splits the secret s into a guessed part and a remainder. By iterating through all possible values of the guessed part (prioritizing values near zero due to the CBD distribution), it reduces the dimension of the lattice embedding. A smaller lattice dimension exponentially decreases the cost of the subsequent LLL/BKZ reduction.

Python

```
def run_hybrid_attack(A, b, q, true_s, true_e, eta,
guess_dim=1, beta=10):
    m, n = A.shape
    n_rem = n - guess_dim
    A_rem = A[:, :n_rem]
    A_guess = A[:, n_rem:]
    cbd_range = list(range(-eta, eta + 1))
    cbd_range.sort(key=abs)
    guesses = list(itertools.product(cbd_range,
repeat=guess_dim))
    for guess in guesses:
```

```
        guess_vec = np.array(guess, dtype=np.int64)
        b_prime = (b - A_guess @ guess_vec) % q
        B = build_embedding_lattice(A_rem, b_prime, q)
        LLL.reduction(B)
        if beta > 2:
            params = BKZ.Param(block_size=beta, max_loops=5)
            BKZ.reduction(B, params)
        # Check recovery...
```

4. Progressive BKZ with Auto-Abort & Pruning

Description & Details: This snippet demonstrates the cost-optimization logic for the BKZ attack. Rather than brute-forcing block sizes, it uses the Geometric Series Assumption (GSA) to predict the theoretical block size beta required to recover the secret. It then sweeps a narrow window around this beta. Furthermore, it utilizes fpylll's pruned BKZ with AUTO_ABORT enabled, which aggressively terminates reduction loops that aren't converging, drastically saving wall-clock time during the sweep.

Python

```python
sigma = math.sqrt(eta / 2.0)
gsa_b = predicted_beta(N, q, d, sigma)
sweep_betas = []
if gsa_b is None or gsa_b <= 20:
    sweep_betas = list(range(2, 42, 2))
else:
    sweep_betas = [max(4, gsa_b - 4), gsa_b, gsa_b + 4]

for b_size in sweep_betas:
```

```
    params = BKZ.Param(
        block_size=b_size,
        float_type="mpfr",
        max_loops=10,
        flags=BKZ.AUTO_ABORT,
        prune=10)
    BKZ.reduction(B, params)
```

5. Sieving Attack via General Sieve Kernel (g6k)

Description & Details: This represents the asymptotically fastest attack vector implemented in the project. It utilizes the g6k (General Sieve Kernel) library to perform GaussSieve on the Kannan-embedded lattice. After LLL pre-reduction, the siever populates a database of short vectors. The assertion verifies that the siever successfully found vectors shorter than or equal to the LLL baseline, from which the target (e,-s,1) vector is extracted.

Python

```
def run_sieve_attack(A, b, q, true_s, true_e):
    B = build_embedding_lattice(A, b, q)
    d = B.nrows
    LLL.reduction(B)
    lll_min_norm = np.linalg.norm([B[0, j] for j in range(d)])
    siever = Siever(B)
    siever.initialize_local(0, 0, d)
    siever.gauss_sieve()
    B_np = np.array([[B[i, j] for j in range(d)] for i in
range(d)], dtype=np.int64)
    candidates = [B_np[i].copy() for i in range(d)]
```

```
    for c in siever.itervalues():
        candidates.append(np.dot(c, B_np))
    sieve_norms = [np.linalg.norm(v) for v in candidates if
np.any(v)]
    sieve_min_norm = min(sieve_norms) if sieve_norms else
lll_min_norm
    assert sieve_min_norm <= lll_min_norm + 1e-9
```

6. Empirical vs. Theoretical Analysis Pipeline

Description & Details: This snippet drives the analytical output of the project. Using Pandas and Matplotlib, it aggregates the raw CSV data generated by the sweep runner. It isolates the block sizes beta at which BKZ and Pruned-BKZ empirically succeeded in recovering the key, and plots them against the theoretical beta predicted by the Geometric Series Assumption (GSA).

Python
```
def plot_empirical_vs_gsa(df):
    """Plots empirical block size beta vs GSA theoretical
predictions."""
    bkz_df = df[df["method"].isin(["primal_bkz",
"pruned_bkz"])]
    grouped = bkz_df.groupby(["lattice_dim",
"method"]).first().reset_index()
    dims = sorted(grouped["lattice_dim"].unique())
    plt.plot(dims, gsa_betas, label="GSA Predictor")
    plt.plot(dims, primal_betas, label="Empirical BKZ")
    plt.plot(dims, pruned_betas, label="Empirical Pruned BKZ")
```

For full source code: https://github.com/havishdgv13-ux/mlwe-cryptanalysis/